MACHINE LEARNING IN PHYSICS
FOUNDATIONS 2

HARRISON B. PROSPER

PHY6937 / PHY4936

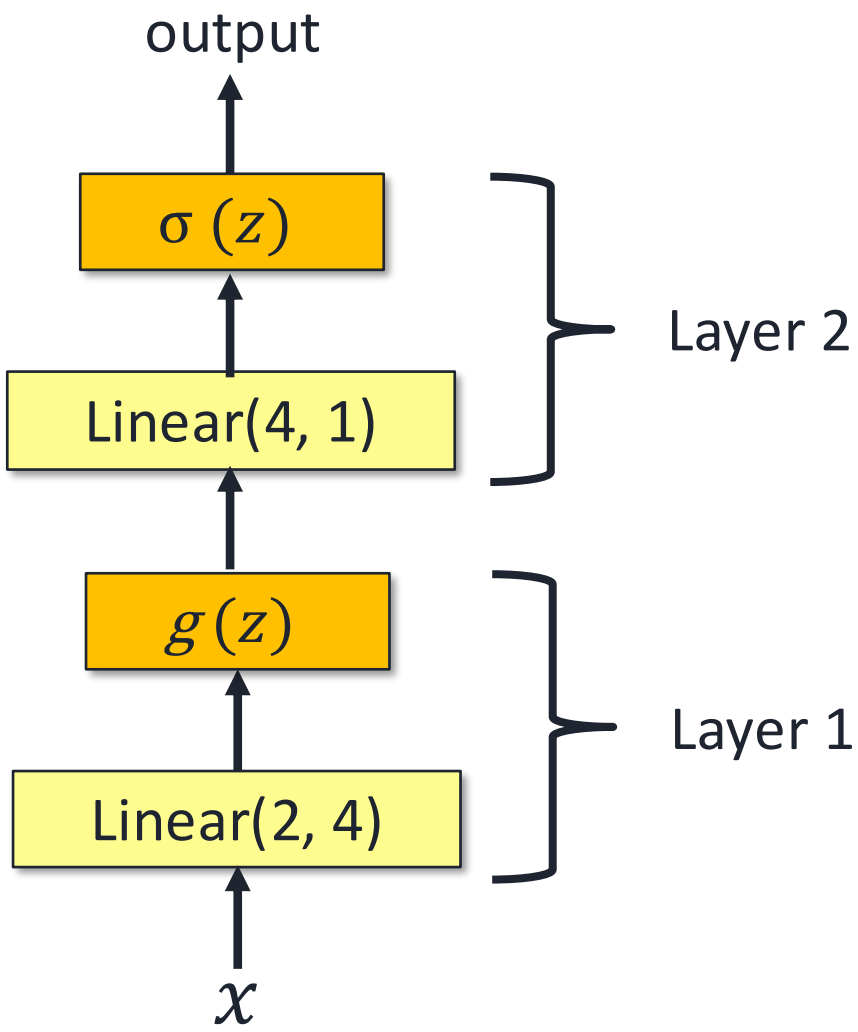
Recap

Last time we introduced the multi-node, multi-layer perceptron (MLP), which is also known as a **feed-forward neural network** (FFNN).

$\sigma(z) = (1 + e^{-z})^{-1}$

$g(z) = \text{relu}(z)$

$h(z) = zA^T + b$



FITTING MODELS TO DATA

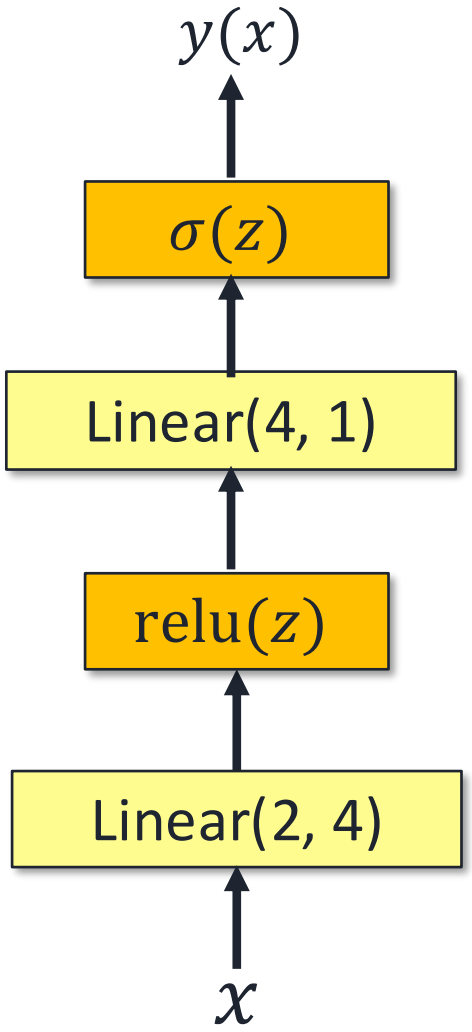
Fitting Models to Data (1)

Suppose our goal is to approximate a function

$$y = f(x_1, x_2)$$

given a dataset $D = \left\{(x_1, x_2, y)_{i=1}^{N}\right\}$.

The standard approach is to approximate $f(x_1, x_2)$ with a *parameterized* function $f_\omega(x_1, x_2)$, that is, a **model**, where ω are the parameters of the model.



Fitting Models to Data (2)

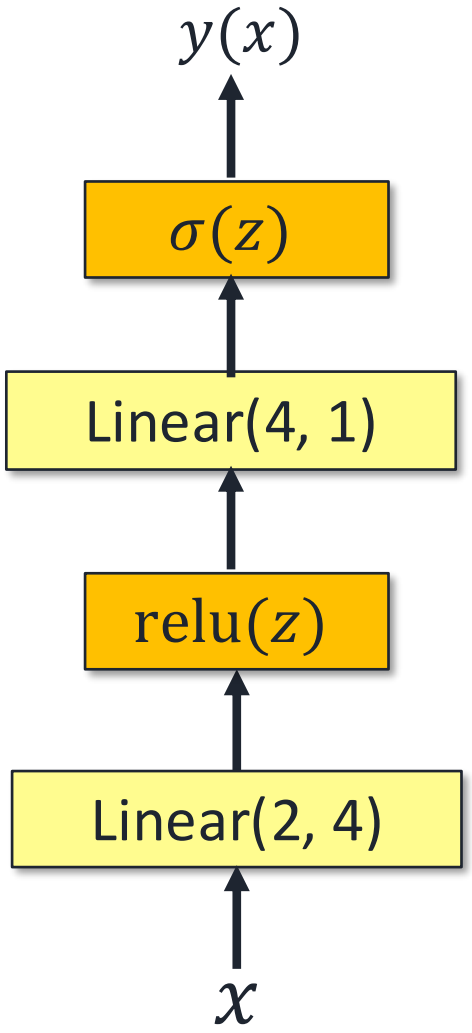
One way to find the *best-fit values* ω^* of the model $f_\omega(x_1, x_2)$ is to minimize the **quadratic form**

$$R(\omega) = \frac{1}{N}\sum_{i=1}^{N}(y_i - f_i)^2$$

with respect to the parameters ω, where

$$f_i \equiv f_\omega(x_{1i}, x_{2i}).$$

This is called a **least-squares fit**.



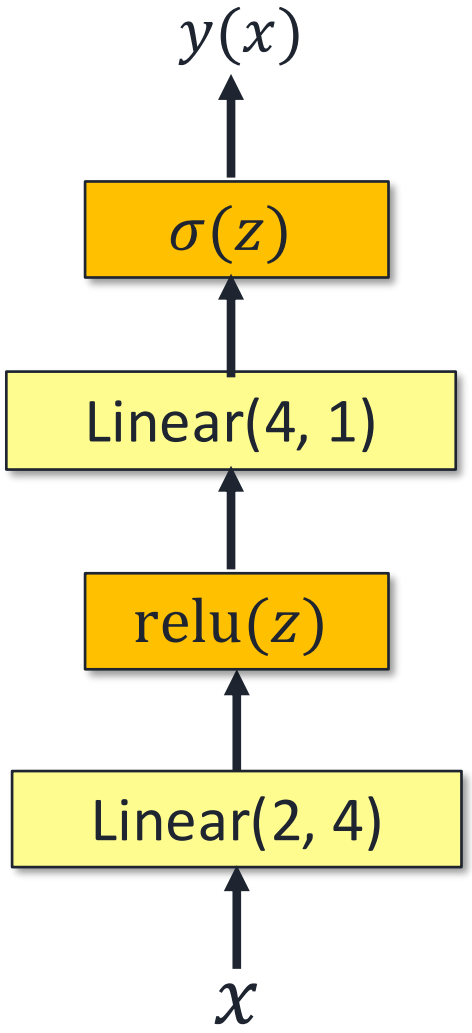
Fitting Models to Data (3)

The least-squares fit can be generalized to

$$R(\omega) = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f_i)$$

where, $L(y_i, f_i)$, the **loss function**, measures the discrepancy between the desired **target** y and the model, f.

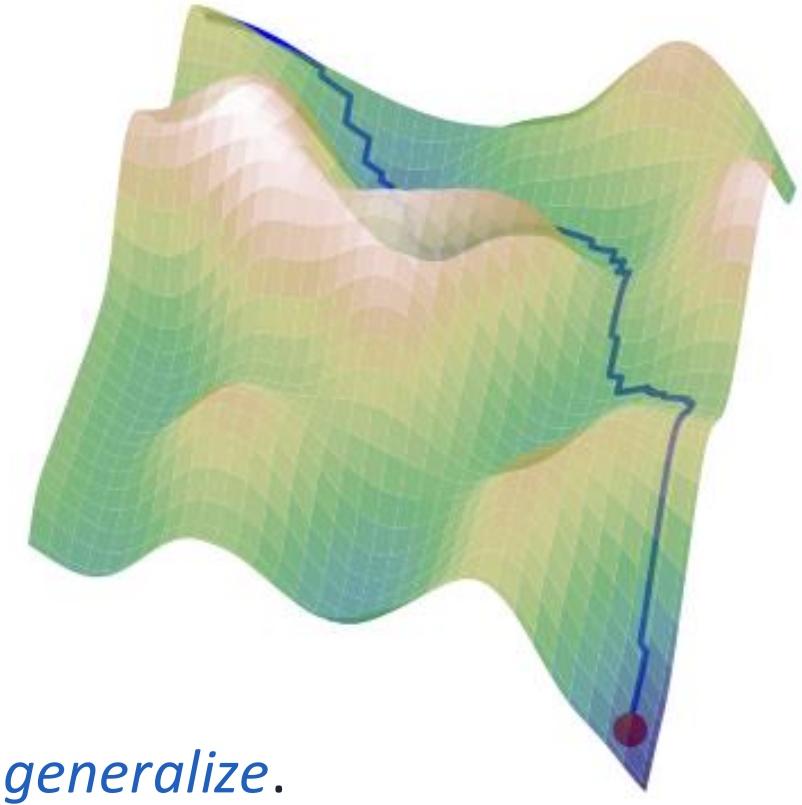
$R(\omega)$ is called the **empirical risk** or **average loss**.
Warning: In ML, the empirical risk is often referred to as the "loss".



Empirical Risk Landscape (1)

The empirical risk defines a highly corrugated high-dimensional “landscape” in the space of parameters.

Our task is to navigate the empirical risk landscape to find a *good approximation* of the minimum not of *that* landscape but rather of the landscape that would be formed if we had an *infinite* amount of data.

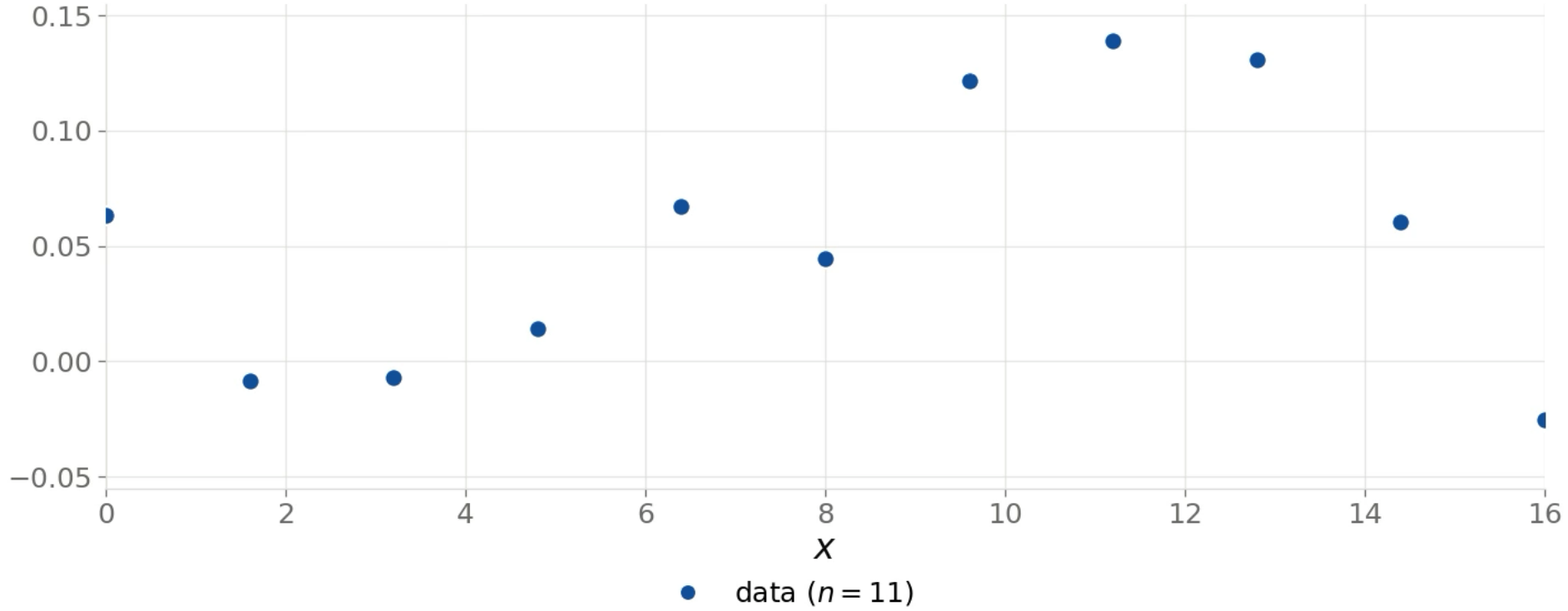


A model that approximates this ideal is said to *generalize*.

Empirical risk is what we can measure — generalization is what we want

We only ever see a finite, noisy sample.

Everything we compute is built from these 11 points.



Example: A least squares fit to a **linear model**.

$$f(x,\omega) = \sum_{j=0}^{n} \omega_j g_j(x)\,, \quad g_j(x) = x^j$$

THE RISK FUNCTIONAL

Risk Functional (1)

In mathematics and physics, insight can often be gained by taking some limit of an expression. Consider the limit of

$$R(\omega) = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f_i)$$

as $N \to \infty$, that is, as the amount of data grows without limit.

In that limit, the empirical risk becomes the **risk functional***,

$$R[f] = \int dx \int dy\, L(y, f)\, p(x, y)$$

where $p(x, y)dxdy$ is the probability distribution of the data.

*A functional depends on all values of a function simultaneously.

Risk Functional (2)

To find the optimal function $f = f^*$, ideally, we should minimize the risk functional

$$R[f] = \int dx \int dy\, L(y, f)\, p(x, y)$$

Of course, in practice, we minimize the empirical risk $R(\omega)$ because we don't have an infinite amount of data.

However, since $R(\omega)$ is a **Monte Carlo** approximation of $R[f]$, any result derived for the risk functional pertains, at least *approximately*, to the empirical risk.

Risk Functional (3)

To minimize

$$R[f] = \int dx \int dy\, L(y, f)\, p(x, y)$$

first note that $p(x, y) = p(y|x)\, p(x)$.

Therefore, we can write the functional $R[f]$ as

$$R[f] = \int dx\ p(x)\, \boldsymbol{\mathcal{L}(x, f)}$$

where,

$$\boldsymbol{\mathcal{L}(x, f)} = \int dy\, L(y, f)\, p(y|\, x)$$

Risk Functional (4)

Add an *arbitrary* function $\epsilon g(x)$ to the optimal function f^*, where ϵ is an infinitesimal*.

$$R[f^* + \epsilon g] = \int dx\ p(x)\ \boldsymbol{\mathcal{L}(x, f^* + \epsilon g)}$$

$$= \int dx\ p(x) \left(\boldsymbol{\mathcal{L}(x, f^*)} + \boldsymbol{\epsilon g} \frac{\partial \boldsymbol{\mathcal{L}}}{\partial f^*} \right)$$

$$= R[f^*] + \boldsymbol{\epsilon} \int dx\ p(x) \boldsymbol{g(x)} \frac{\partial \boldsymbol{\mathcal{L}}}{\partial f^*}$$

* $0 < |\epsilon| < |x|$, where x is any real number.

Risk Functional (5)

Rearranging, and taking the standard part* of both sides, we find

$$\mathrm{st}\left(\frac{R[f^* + \epsilon g] - R[f^*]}{\epsilon}\right) = \int dx\, p(x) g(x) \frac{\partial \mathcal{L}}{\partial f^*}$$

The lefthand side *defines* the **functional derivative** $\delta R / \delta f$.

By assumption, $\delta R / \delta f$ is zero at $f = f^*, \ \forall\, x$. Therefore,

$$\int dx\, p(x) g(x) \frac{\partial \mathcal{L}}{\partial f} = 0$$

when $f = f^*$.

*The standard part of the hyperreal $x^* = x + \epsilon a$ is defined by $x = \mathrm{st}(x^*)$

Risk Functional (6)

We want the expression

$$\int dx\, p(x) g(x) \frac{\partial \mathcal{L}}{\partial f} = 0$$

to hold for *any* function $g(x)$ and $\forall\, x$.

This can happen *if only if*

$$\frac{\partial \mathcal{L}}{\partial f} = 0$$

that is,

$$\frac{\partial \int dy\, L(y, f)\, p(y|\, x)}{\partial f} = 0$$

Risk Functional (7)

If $\partial/\partial f$ and $\int$ commute, we obtain the very important result:

$$\int dy \, \frac{\partial L}{\partial f} p(y \mid x) = 0 \rightarrow \mathbb{E}_{y \sim p(y|x)} \left[\frac{\partial L}{\partial f} \right] = 0$$

1. The result is *independent* of the details of $f_\omega(x)$. However,…
2. The function $f_\omega(x)$ must have sufficient **capacity**: i.e., there must exist an approximation $f_{\hat{\omega}}(x)$ found by minimizing the empirical risk $R(\omega)$ that is close (in some precise sense) to the optimal function f^*.
3. Moreover, it must be possible to find that function!

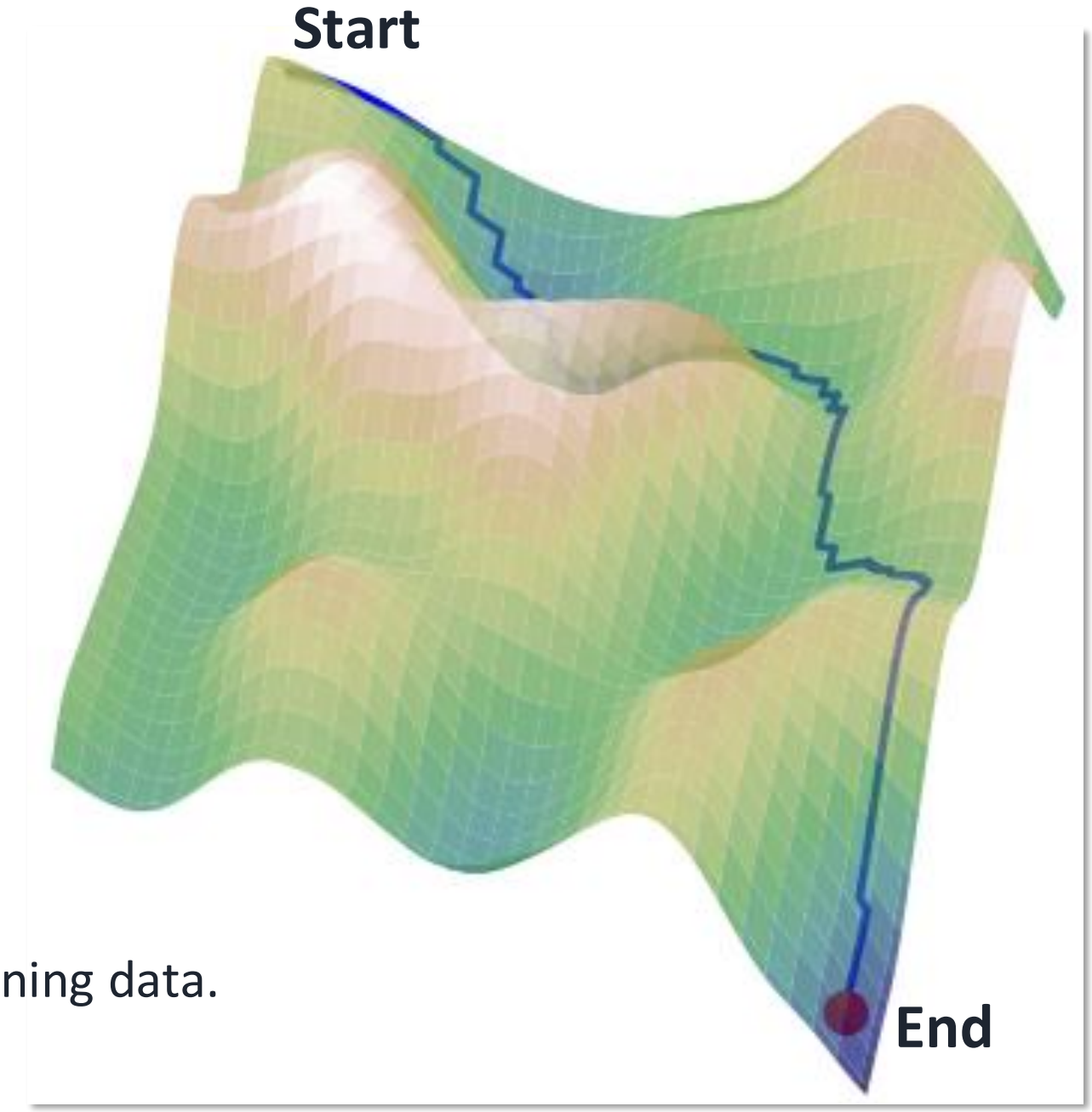
EMPIRICAL RISK MINIMIZATION

Train: fit a model to (training) data.

Batch: a subset of the training data.

Iteration: a single optimization step.

Epoch: a single pass through the entire training data.



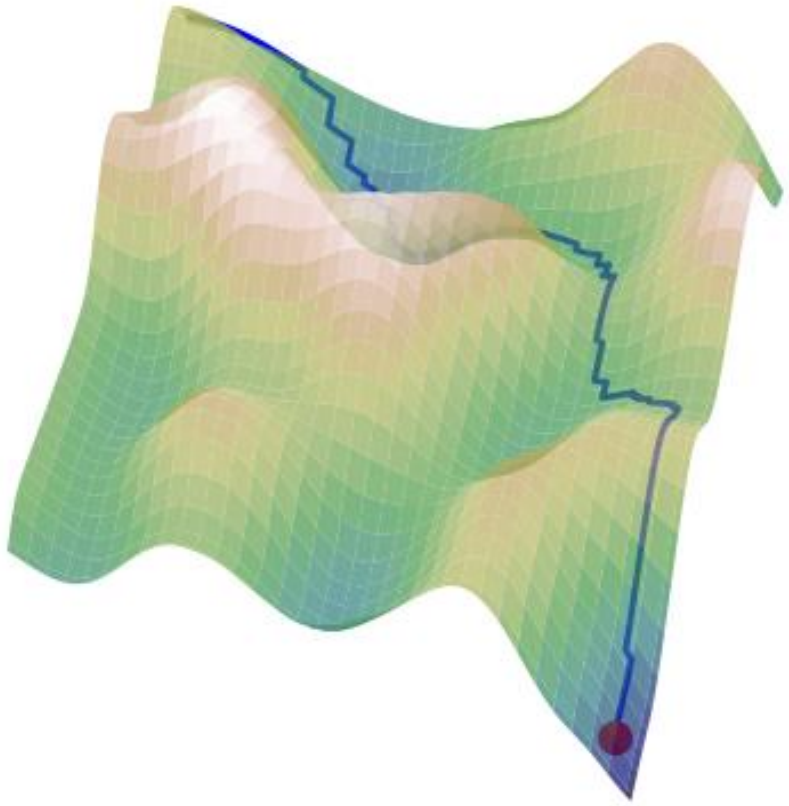
Empirical Risk Minimization (1)

The minimization of $R(\omega)$ is typically done by moving in the direction of *steepest descent*. Most optimization algorithms in ML are variations of **S**tochastic **G**radient **D**escent.

Algorithm

1. At the current point ω_j, compute a *noisy* approximation of the gradient of $R(\omega) \approx \frac{1}{n}\sum_{i=1}^{n} L(y_i, f_i)$ by using a batch of training data of size $n \ll N$.
2. Move to position ω_{j+1} in the landscape using

$$\omega_{j+1} = \omega_j - \eta \nabla R$$



Empirical Risk Minimization (2)

Why does this (1st-order) algorithm

$$\omega_{j+1} = \omega_j - \eta \nabla R$$

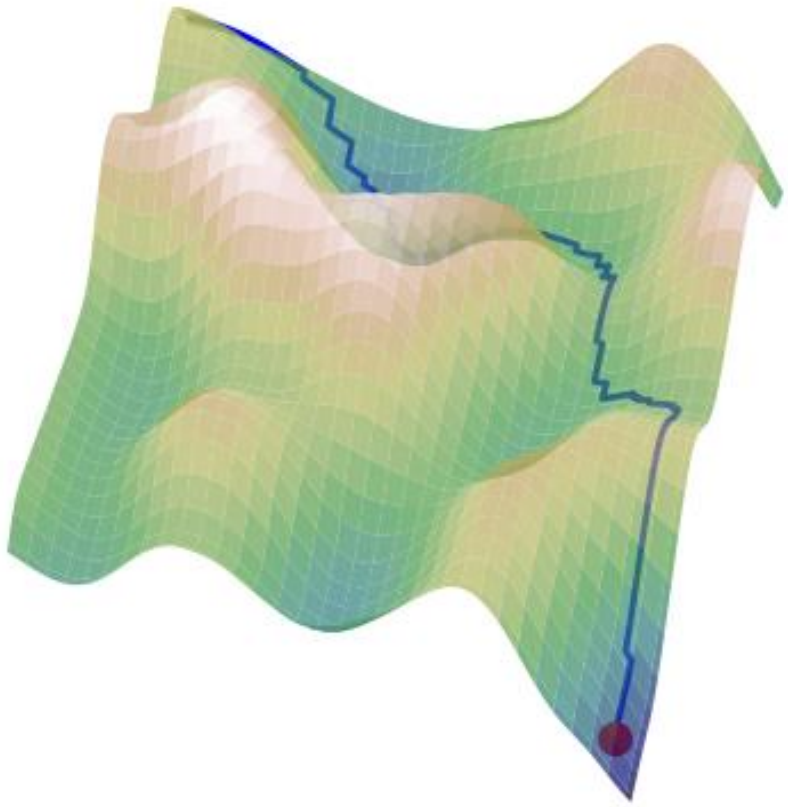
work? (η is called the **learning rate**.)

Consider

$$R(\omega_{j+1}) = R(\omega_j - \eta \nabla R)$$
$$= R(\omega_j) - \eta \nabla R \cdot \nabla R + O(\eta^2)$$

If the $O(\eta^2)$ can be neglected, and given that the $O(\eta)$ term is always negative, it follows that

$$R(\omega_{j+1}) < R(\omega_j).$$



Summary

Risk Minimization

- Given a data set $D = \{(x, y)\}_{i=1}^{N}$, a model $f_\omega(x)$, and a loss function $L(y, f)$, the optimal function $f_{\hat{\omega}}(x)$ satisfies:

$$\int \frac{\partial L}{\partial f} p(y \mid x) dy = 0$$

  at least *approximately*.

Empirical Risk Minimization

- Many minimization algorithms are variations of the following algorithm:

$$\omega_{j+1} = \omega_j - \eta \nabla R$$

- Batches of data are used, which introduces noise into ∇R.